

Signalpost Norway Agent

Builderr.ai challenge — **Signalpost company research**

Open: Aug 23 – Oct 21, 2026 · Prize pool: \$2,500 · Bonus: JBOX \$250/\$150/\$100 · Fortnightly vote: \$100 x 4

What this is

An autonomous agent that builds verified company-intelligence profiles for Norway-registered companies from public sources only — official filings, company websites, LinkedIn, job boards, and dated news/public activity. Think "Crunchbase for Norway," rebuilt from scratch with provenance on every claim.

Full challenge context lives in docs/challenge_brief_summary.md.

Repo layout

```
signalpost-norway-agent/
■■■ README.md                ← you are here
■■■ PROJECT_PLAN.md          ← phased build plan, milestones, timeline
■■■ TODO.md                  ← running checklist
■■■ NOTES.md                 ← daily working log
■■■ CHANGELOG.md
■■■ CONTRIBUTING.md          ← workflow, style, commit conventions
■■■ SECURITY.md               ← secrets handling, safe-URL policy
■■■ LICENSE                  ← MIT (adjust if the challenge requires otherwise)
■■■ Makefile                  ← install / lint / test / crawl-dev / freeze
■■■ pyproject.toml           ← ruff + pytest config
■■■ requirements.txt / requirements-dev.txt
■■■ Dockerfile / docker-compose.yml ← reproducible batch runs
■■■ .env.example              ← required env vars (no secrets committed)
■■■ .editorconfig / .gitignore
■■■ .github/
■   ■■■ workflows/ci.yml      ← lint + test + idempotency check on every push
■   ■■■ PULL_REQUEST_TEMPLATE.md
■   ■■■ ISSUE_TEMPLATE/      ← bug report, strategy/feature idea
■■■ docs/
■   ■■■ challenge_brief_summary.md ← condensed rules, gates, scoring, budget
■   ■■■ architecture.md / architecture.mmd ← system design (text + Mermaid diagram)
■   ■■■ data_schema.md        ← output-contract doc (mirrors src/validate/schema.py)
■■■ source_policy.md          ← what we're allowed to crawl and how
■■■ scoring_and_gates.md      ← hard gates + scoring weights, mapped to our checks
■■■ learning_harness.md       ← strategy try/score/promote/freeze loop
■■■ api_and_budget.md         ← request/time/cost budget tracking per batch
■■■ SUBMISSION.md            ← fill-in template for each submission/revision
■■■ glossary.md / faq.md / risk_register.md
■■■ pdf/                      ← PDF exports of the plan + brief summary
■■■ src/
■   ■■■ config.py             ← env-driven settings, single source of truth for budgets/keys
■   ■■■ resolve/              ← org number -> canonical entity resolution
■   ■■■ crawl/                ← Scrapy spiders, sitemap discovery, Playwright fallback
■   ■■■ extract/              ← extract (structured data) + Trafilatura (text)
■   ■■■ match/                ← RapidFuzz candidate matching (site/LinkedIn/jobs/news)
```

```

■ ■■■ validate/
■ ■ ■■■ schema.py          ← real Pydantic models (Claim, Entity, CompanyProfile, ResultEnvelope)
■ ■■■ storage/
■ ■ ■■■ snapshot.py        ← append-only snapshot write/read + diff + idempotency check
■ ■■■ api/
■ ■ ■■■ main.py            ← minimal FastAPI app serving profiles
■ ■■■ cli/                 ← run commands: crawl-batch, score-batch, freeze
■■■ frontend/              ← React dashboard scaffold (Phase 8, lowest priority)
■■■ data/
■ ■■■ raw/ processed/ snapshots/ ← gitignored, generated at runtime
■ ■■■ samples/sample_profile.json ← one hand-written example matching the schema
■■■ tests/
■ ■■■ conftest.py          ← shared fixtures
■ ■■■ test_schema.py       ← output-contract validation tests
■ ■■■ test_idempotency.py  ← guards the idempotent-refresh hard gate
■■■ scripts/               ← one-off utility scripts (universe download, sampling)

```

Quick start

```

make dev-install      # installs deps + Playwright browser
cp .env.example .env  # fill in any API keys
make check            # lint + tests
make crawl-dev        # run pipeline against the dev slice (once implemented)

```

Or via Docker: `docker compose run agent`.

Status

Planning + scaffold stage. Core pipeline modules (resolve, crawl, extract, match) are stubs; the schema, storage/diffing logic, and tests are real and reviewed but **not yet run** in this environment (no network access here to install dependencies — run `make check` in your own environment to confirm before building further). See `TODO.md` for the current checklist and `PROJECT_PLAN.md` for the phased build order.